R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Systèmes d'exploitation Unix/Linux, processus, multitâche préemptif et communication inter-processus par signaux

Synthèse exécutive : Ce cours pose les bases fondamentales de la gestion des tâches au sein des systèmes d'exploitation de type Unix/Linux. Il détaille la transition critique entre un programme statique stocké sur disque et un processus dynamique résidant en mémoire RAM. Pour permettre l'exécution simultanée de multiples applications sur un même ordinateur, le noyau s'appuie sur le multitâche préemptif et un modèle d'états à trois niveaux (Élu, Prêt, Bloqué). La séance explicite également l'arborescence généalogique des processus (PID/PPID) issue du processus initial systemd/init (PID 1), le fonctionnement des appels système fork() et exec(), ainsi que le rôle des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT) pour le contrôle et la communication inter-processus.

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour qu'un ordinateur exécute du code, le système d'exploitation doit transformer un programme (données passives sur disque) en un processus (entité active en mémoire RAM). L'ordonnanceur du noyau orchestre cette activité via un modèle à trois états fondamentaux. Lorsqu'un processus est à l'état 'Élu', il occupe physiquement le processeur. Si son quantum de temps CPU expire, le noyau le bascule à l'état 'Prêt' et attribue le CPU à un autre processus. Si le processus déclenche une lecture sur disque ou attend une saisie clavier, il bascule à l'état 'Bloqué' pour ne pas gaspiller de cycles CPU. Dès que la ressource est disponible, le noyau le place dans la file d'attente à l'état 'Prêt'. Tous les processus sous Unix dérivent d'un arbre généalogique strict démarrant au processus racine PID 1 (systemd/init).

- Programme vs Processus : Un programme est un fichier exécutable passif sur disque ; un processus est son instance dynamique en RAM avec des ressources allouées.
- Multitâche préemptif : Mécanisme où l'ordonnanceur du noyau alloue et retire dynamiquement des tranches de temps CPU (quanta) aux processus.
- Bloc de Contrôle du Processus (PCB) : Structure interne du noyau regroupant PID, PPID, état, registres CPU, carte mémoire et UID.
- Arborescence PID/PPID : Filiation des processus dont le processus racine utilisateur est systemd ou init (PID 1).
- Appels système fork() et exec() : fork() duplique le processus parent, puis exec() remplace le code de l'enfant par un nouveau programme.
- Trois états d'un processus : Élu (exécute sur le CPU), Prêt (attend du temps CPU), Bloqué (en attente d'une E/S ou d'un événement externe).

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme sur disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (statique, passif, ne consommant ni énergie ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, consommant de l'espace sur le plan de travail/RAM, du temps de cuisson/CPU et manipulant des ustensiles/descripteurs de fichiers).*

Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux représentent la forme de communication inter-processus (IPC) la plus élémentaire et asynchrone dans les systèmes POSIX. Lorsqu'un signal est émis vers un processus, le noyau interrompt son exécution normale pour exécuter le gestionnaire associé. Un développeur peut programmer son application pour intercepter (catch) ou ignorer (ignore) la plupart des signaux. Cependant, la norme POSIX garantit l'autorité absolue de l'administrateur système en rendant deux signaux strictement incapturables et inignorables : SIGKILL (9) et SIGSTOP (19), traités

en direct par le noyau.

• Signal POSIX : Interruption logicielle asynchrone transmise à un processus pour l'informer d'un événement ou lui imposer une instruction.

• Gestionnaire de signal (Signal Handler) : Fonction exécutée par un processus pour intercepter (catch) un signal et en personnaliser le comportement.

• SIGINT (Signal 2) : Émis via 'Ctrl + C' ; demande l'interruption douce d'une tâche interactive.

• SIGTERM (Signal 15) : Signal de terminaison par défaut de 'kill' ; invite le processus à libérer ses ressources et s'arrêter proprement.

• SIGKILL (Signal 9) : Ordre d'extinction inconditionnel géré directement par le noyau pour détruire immédiatement le processus sans sommation.

• Job Control (SIGSTOP 19 / SIGCONT 18) : 'Ctrl + Z' envoie SIGSTOP pour suspendre un processus ; 'fg' ou 'bg' envoie SIGCONT pour reprendre son exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à la porte pour vous demander de quitter les lieux (vous avez le temps de ranger vos affaires, sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (9) est une intervention d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Parce que sans le tiret ('-'), le shell interprète '9' comme un argument positionnel (le PID du processus à cibler) et cherche un autre argument pour le signal, ou omet le signal par défaut. Le tiret est indispensable pour indiquer à la commande 'kill' qu'il s'agit d'un numéro de signal et non d'un PID.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus zombie est un processus qui a terminé son exécution mais dont l'entrée est toujours présente dans la table des processus du noyau car son parent n'a pas encore lu son code de retour (via l'appel système wait()). Il est impossible de tuer un zombie avec la commande 'kill' (même avec SIGKILL) puisqu'il est déjà mort ; la seule solution est de nettoyer ou de stopper son processus parent pour que le noyau récupère les ressources.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche l'identifiant du job ainsi que le PID attribué par le système (ex: [1] 4521) et rend immédiatement l'invite de commande à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' liste l'ensemble des processus du système et filtre la sortie pour repérer le processus 'sleep'. Elle permet de vérifier son PID (4521) et son PPID (qui correspond au PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus. Comme l'utilitaire 'sleep' ne possède pas de gestionnaire personnalisé interceptant ce signal, l'action par défaut du système s'applique : le processus interrompt proprement son exécution et se termine.

Résultat : Le processus sleep est définitivement arrêté. Une nouvelle exécution de 'ps -ef | grep sleep' confirme sa disparition totale de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le processus racine est systemd (ou init) et son PID est toujours 1.

Q : Quel signal POSIX est transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' dans son terminal ?

R : Il s'agit du signal SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté ni bloqué par un programme informatique ?

R : Parce qu'il est traité directement par le noyau du système d'exploitation pour garantir une porte de sortie administrative absolue en cas de blocage critique.